



Swinburne University of Technology Hawthorn Campus
Department of Computing Technologies

COS20028 Big Data Architecture and Application
Assignment 1 - Semester 2, 2025

Student Name: [NGUYEN PHAM DUY](#)

Student ID: [104058375](#)

Assessment Title: MapReduce Application.

Assessment Outcome: 20%

Due Date: AEST 23:59 on 11/06/2025.

Assessable Item:

- One (1) piece of a report containing the complete flowchart showing your design idea about how to prepare the data, the tools being used in all steps, and the report for answers to the given questions.
- One (1) piece of a video explaining how you found answers to the question, the challenges you encountered, and how you solved them.

-
1. Correct implementation of the Mapper class(es) with readability and well-structured codes and comments. **[20 marks]**

“**WordMapper**” is the mapper for the first MapReduce job. It filters words from the dataset that begin with an uppercase letter from the set {A, C, G, W, Y} and are followed by lowercase letters. It uses the regular expression “`\\b[ACGWY][a-z]+\\b`” to perform the match and emits each valid word with a count of one.

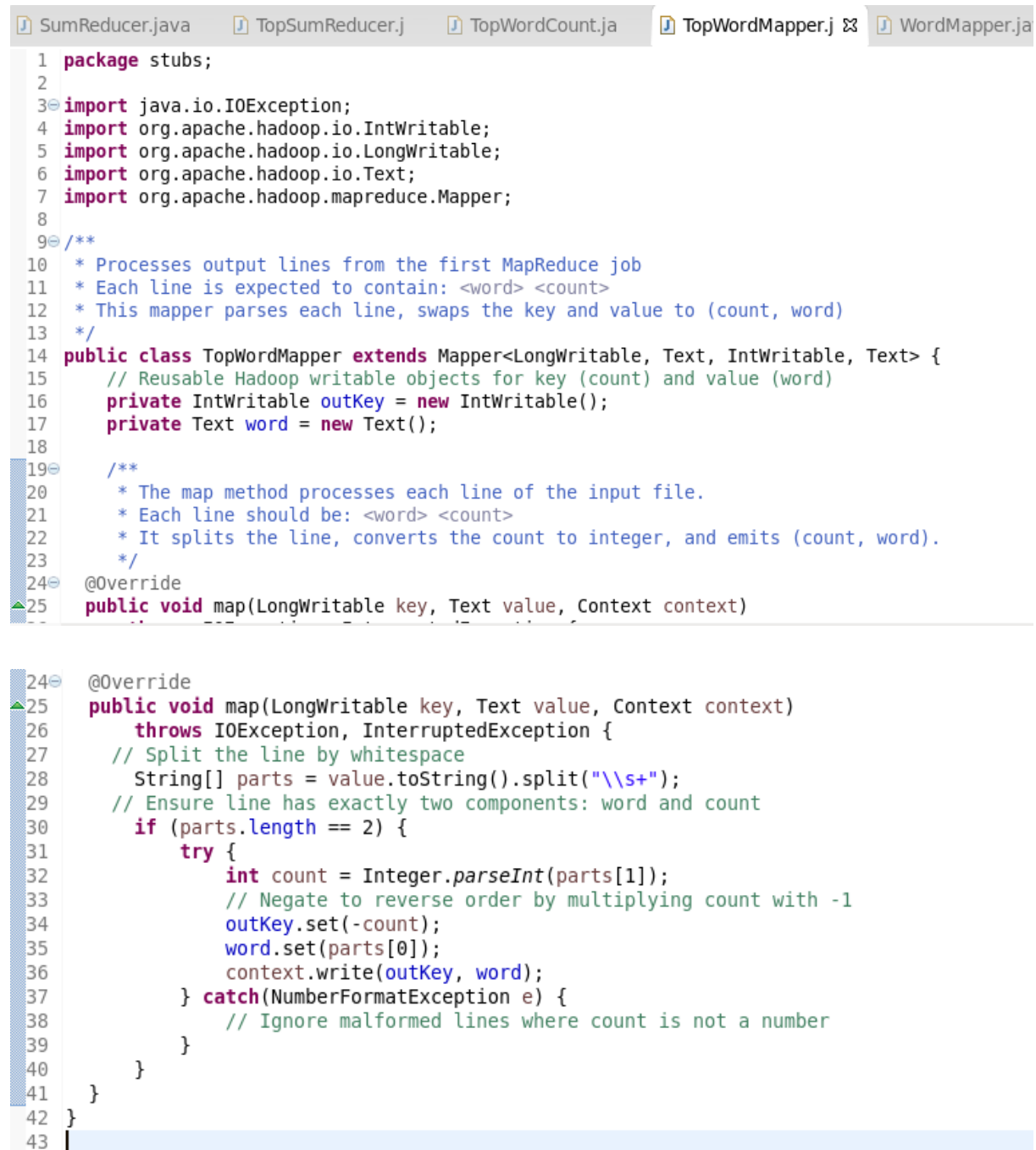
```

*WordMapper.java  WordCount.java  SumReducer.java  TopSumReducer.j  TopWordCount
1 package stubs;
2
3 import java.io.IOException;
11
12 /**
13  * WordMapper filters and emits words that:
14  * - Start with one of the uppercase letters: "A,C,G,W,Y"
15  * - Followed by lowercase letters
16  *
17  * For each matching word, this mapper emits (word, 1) as output
18  *
19  */
20 public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
21     //Reusable Text object to avoid creating new objects in loop
22     private Text word = new Text();
23
24     //Regex Pattern
25     private Pattern pattern = Pattern.compile("\\b[ACGWY][a-z]+\\b");
26
27 /**
28  * The map method is called once per line of input.
29  * It finds all words matching the defined pattern and emits them with count 1.
30  */
31 @Override
32 public void map(LongWritable key, Text value, Context context)
33
34 @Override
35 public void map(LongWritable key, Text value, Context context)
36     throws IOException, InterruptedException {
37     // Apply regex matcher to the line
38     Matcher matcher = pattern.matcher(value.toString());
39     // For each match, emit the word with count 1
40     while (matcher.find()) {
41         // Extract matched word
42         word.set(matcher.group());
43         // Emit (word, 1)
44         context.write(word, new IntWritable(1));
45     }
46 }
47 }
48 }
49 }
50 }
51 }
52 }
53 }
54 }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }

```

Figure 1. Mapper class for Job 1 – Word Filtering.

“**TopWordMapper**” is used in the second MapReduce job. It reads the output from the first MapReduce job and switches the key-value pairs from (word, count) to (count, word), enabling sorting to arrange the counts in descending order by multiplying the key (count) by -1.



```

1 package stubs;
2
3 import java.io.IOException;
4 import org.apache.hadoop.io.IntWritable;
5 import org.apache.hadoop.io.LongWritable;
6 import org.apache.hadoop.io.Text;
7 import org.apache.hadoop.mapreduce.Mapper;
8
9 /**
10  * Processes output lines from the first MapReduce job
11  * Each line is expected to contain: <word> <count>
12  * This mapper parses each line, swaps the key and value to (count, word)
13  */
14 public class TopWordMapper extends Mapper<LongWritable, Text, IntWritable, Text> {
15     // Reusable Hadoop writable objects for key (count) and value (word)
16     private IntWritable outKey = new IntWritable();
17     private Text word = new Text();
18
19     /**
20      * The map method processes each line of the input file.
21      * Each line should be: <word> <count>
22      * It splits the line, converts the count to integer, and emits (count, word).
23      */
24     @Override
25     public void map(LongWritable key, Text value, Context context)
26         throws IOException, InterruptedException {
27         // Split the line by whitespace
28         String[] parts = value.toString().split("\\s+");
29         // Ensure line has exactly two components: word and count
30         if (parts.length == 2) {
31             try {
32                 int count = Integer.parseInt(parts[1]);
33                 // Negate to reverse order by multiplying count with -1
34                 outKey.set(-count);
35                 word.set(parts[0]);
36                 context.write(outKey, word);
37             } catch (NumberFormatException e) {
38                 // Ignore malformed lines where count is not a number
39             }
40         }
41     }
42 }
43

```

Figure 2. Mapper class for Job 2 – Key-Value Swapping and Sorting Descending Order.

2. Correct implementation of the Reducer class(es) with readability and well-structured codes and comments. [20 marks]

“**SumReducer**” serves as the reducer in the first MapReduce job. It aggregates the values (counts) associated with each word emitted by the mapper and calculates the total frequency of each word.

```

*SumReducer.jav  TopSumReducer.j  TopWordCount.ja  TopWordMapper.j  WordMapper.java
1  package stubs;
2
3  import java.io.IOException;
4
5  /**
6   * SumReducer receives (word, list of counts) and sums them.
7   *
8   * This is used in the first job to count total occurrences of each word.
9   */
10 public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
11     /**
12      * The reduce method is called once for each unique key (word).
13      * It sums up all the integer values
14      */
15     @Override
16     public void reduce(Text key, Iterable<IntWritable> values, Context context)
17         throws IOException, InterruptedException {
18         // Sum all the values associated with this word
19         for (IntWritable value : values) {
20             wordCount += value.get();
21         }
22         // Emit the word along with its total count
23         context.write(key, new IntWritable(wordCount));
24     }
25 }
26
27
28
29
30
31
32
33
34

```

Figure 3. Reducer class for Job 1 – Word Count Aggregation.

“*TopSumReducer*” class as the reducer in the second job. It receives sorted (count, word) pairs, restores the original positive count, and outputs the two most frequent terms as well as the total number of words that appeared exactly once under the given criteria.

```

*SumReducer.jav  TopSumReducer.j  TopWordCount.ja  TopWordMapper.j  WordMapper.java  SumR
1  package stubs;
2
3  import java.io.IOException;
4
5  /**
6   * TopSumReducer processes (count, [words]) pairs.
7   *
8   * Perform two tasks:
9   * 1. Outputs the top 2 most frequent words with their counts.
10  * 2. Counts and reports how many words occurred exactly once (count == 1).
11  */
12 public class TopSumReducer extends Reducer<IntWritable, Text, Text, IntWritable> {
13     // Keeps track of how many top-ranked words have been output
14     private int rank = 0;
15
16     // Counter for words that appear only once in the dataset
17     private int singleCountWords = 0;
18
19     /**
20      * Called once per unique count (key), with all words (values) that share that count.
21      */
22     @Override
23     public void reduce(IntWritable key, Iterable<Text> values, Context context)
24         throws IOException, InterruptedException {
25         // Negated count, restore the original positive count
26
27
28
29
30
31
32
33
34

```



```

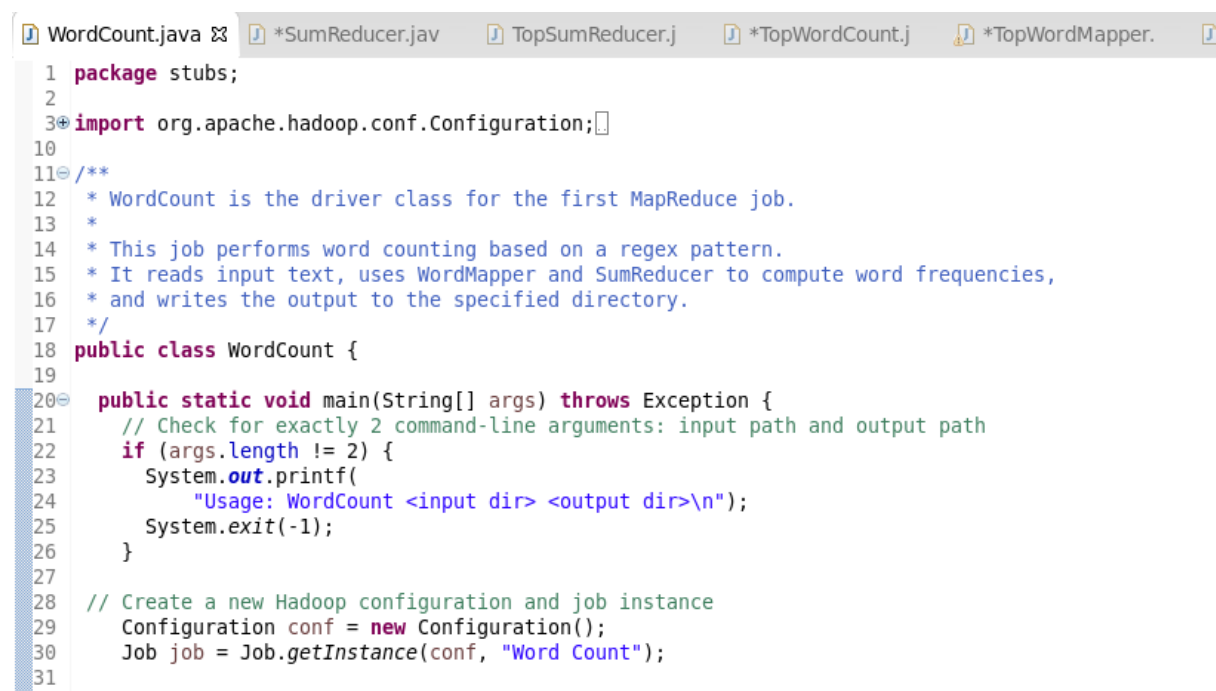
29 // Negated count, restore the original positive count
30 int count = -key.get();
31 for (Text word : values) {
32     // Output the top 2 words with the highest frequencies
33     if (rank < 2) {
34         rank++;
35         context.write(new Text ("Top" + rank + ": " + word.toString()), new IntWritable(count));
36     }
37
38     // Count how many words occurred exactly once
39     if (key.get() == -1) {
40         singleCountWords++;
41     }
42 }
43
44 /**
45  * Called once after all reduce calls are finished.
46  * Outputs the total number of words that occurred only once.
47  */
48 @Override
49 protected void cleanup(Context context) throws IOException, InterruptedException {
50     context.write(new Text("Single-count words"), new IntWritable(singleCountWords));
51 }
52 }
53

```

Figure 4. Reducer class for Job 2 – Extract Top Words and Single-Count Terms.

3. Correct implementation of the Driver class(es) with readability and well-structured codes and comments. [20 marks]

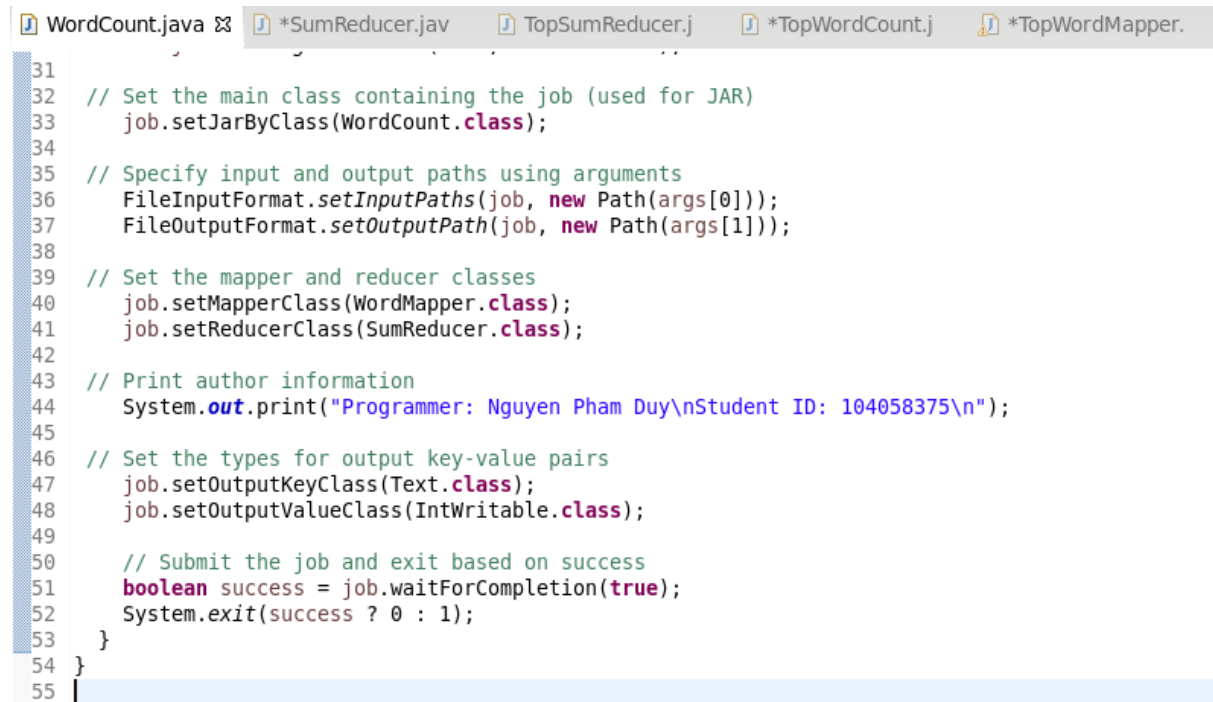
“**WordCount**” is the driver class for the first MapReduce job. It sets up the job configuration, invokes the “**WordMapper**” and “**SumReducer**” classes, and processes the dataset to produce the word frequency list based on the regex criteria.



```

1 package stubs;
2
3 import org.apache.hadoop.conf.Configuration;
4
5
6 /**
7  * WordCount is the driver class for the first MapReduce job.
8  *
9  * This job performs word counting based on a regex pattern.
10  * It reads input text, uses WordMapper and SumReducer to compute word frequencies,
11  * and writes the output to the specified directory.
12  */
13 public class WordCount {
14
15     public static void main(String[] args) throws Exception {
16         // Check for exactly 2 command-line arguments: input path and output path
17         if (args.length != 2) {
18             System.out.printf(
19                 "Usage: WordCount <input dir> <output dir>\n");
20             System.exit(-1);
21         }
22
23         // Create a new Hadoop configuration and job instance
24         Configuration conf = new Configuration();
25         Job job = Job.getInstance(conf, "Word Count");
26
27
28
29
30
31

```



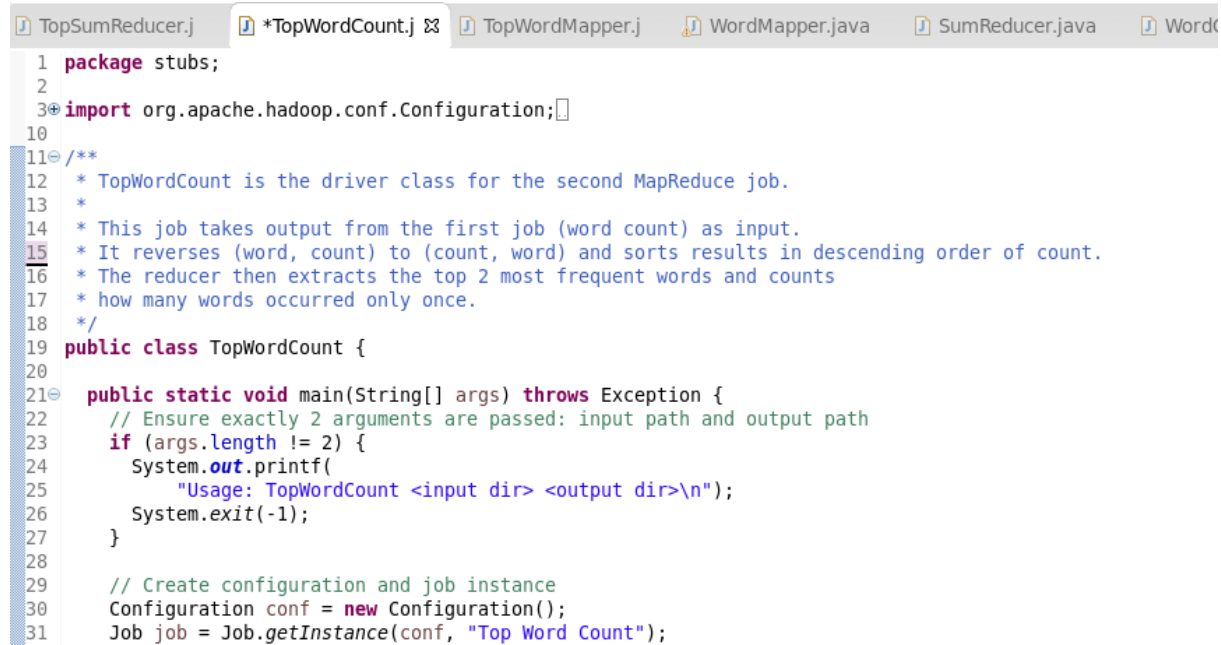
```

31
32 // Set the main class containing the job (used for JAR)
33 job.setJarByClass(WordCount.class);
34
35 // Specify input and output paths using arguments
36 FileInputFormat.setInputPaths(job, new Path(args[0]));
37 FileOutputFormat.setOutputPath(job, new Path(args[1]));
38
39 // Set the mapper and reducer classes
40 job.setMapperClass(WordMapper.class);
41 job.setReducerClass(SumReducer.class);
42
43 // Print author information
44 System.out.print("Programmer: Nguyen Pham Duy\nStudent ID: 104058375\n");
45
46 // Set the types for output key-value pairs
47 job.setOutputKeyClass(Text.class);
48 job.setOutputValueClass(IntWritable.class);
49
50 // Submit the job and exit based on success
51 boolean success = job.waitForCompletion(true);
52 System.exit(success ? 0 : 1);
53 }
54 }
55

```

Figure 5. Driver class for Job 1 – Execution setup for filtering and counting.

“**TopWordCount**” is the driver for the second MapReduce job. It reads the output from the first job, swaps the key-value pair and applies descending sort using “**TopWordMapper**”, and invokes the reducer to produce the final top-2 and single-count word statistics.



```

1 package stubs;
2
3 import org.apache.hadoop.conf.Configuration;
4
5
6
7
8
9
10
11 /**
12  * TopWordCount is the driver class for the second MapReduce job.
13  *
14  * This job takes output from the first job (word count) as input.
15  * It reverses (word, count) to (count, word) and sorts results in descending order of count.
16  * The reducer then extracts the top 2 most frequent words and counts
17  * how many words occurred only once.
18  */
19 public class TopWordCount {
20
21     public static void main(String[] args) throws Exception {
22         // Ensure exactly 2 arguments are passed: input path and output path
23         if (args.length != 2) {
24             System.out.printf(
25                 "Usage: TopWordCount <input dir> <output dir>\n");
26             System.exit(-1);
27         }
28
29         // Create configuration and job instance
30         Configuration conf = new Configuration();
31         Job job = Job.getInstance(conf, "Top Word Count");
32

```

```

28
29 // Create configuration and job instance
30 Configuration conf = new Configuration();
31 Job job = Job.getInstance(conf, "Top Word Count");
32
33 // Set the driver class (for JAR reference)
34 job.setJarByClass(TopWordCount.class);
35
36 // Set input and output paths from command-line arguments
37 FileInputFormat.setInputPaths(job, new Path(args[0]));
38 FileOutputFormat.setOutputPath(job, new Path(args[1]));
39
40 // Set the Mapper and Reducer classes
41 job.setMapperClass(TopWordMapper.class);
42 job.setReducerClass(TopSumReducer.class);
43
44 //Print author information
45 System.out.print("Programmer: Nguyen Pham Duy\nStudent ID: 104058375\n");
46
47 // Set intermediate output types from mapper
48 job.setMapOutputKeyClass(IntWritable.class);
49 job.setMapOutputValueClass(Text.class);
50
51 // Set final output types from reducer
52 job.setOutputKeyClass(Text.class);
53
54
55 // Set final output types from reducer
56 job.setOutputKeyClass(Text.class);
57 job.setOutputValueClass(IntWritable.class);
58
59 // Submit the job and exit based on success
60 boolean success = job.waitForCompletion(true);
61 System.exit(success ? 0 : 1);
62 }
63 }
64

```

Figure 6. Driver class for Job 2 – Setup for sorting and result extraction.

(*)Additional MapReduce Implementation: Counting Terms that start with all upper-case letters and follow by the lower-case letters Appearing Only Once in the Entire Document:

To handle the task of counting how many terms that start with all upper-case letters and follow by the lower-case letters appear only once in the entire dataset, an additional MapReduce workflow is implemented. This process uses two MapReduce jobs:

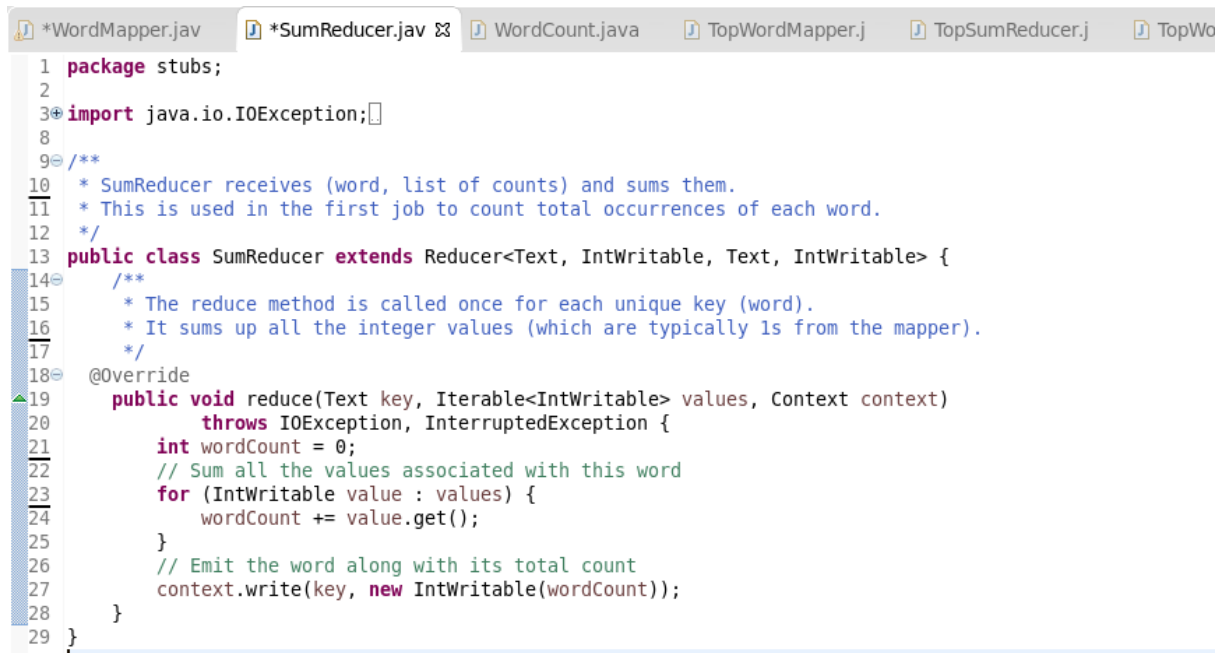
First Job: Word Frequency: The first job uses a mapper that emits all words that satisfy the regular expression “`\\b[A-Z][a-z]+\\b`”. The reducer then sums up the total frequency of each word.


```

*WordMapper.java  SumReducer.java  WordCount.java  TopWordMapper.j  TopSumReduce
1  package stubs;
2
3  import java.io.IOException;
11
12 /**
13  * WordMapper filters and emits words that:
14  * - Start with all of the uppercase letters
15  * - Followed by lowercase letters
16  *
17  * For each matching word, this mapper emits (word, 1) as output
18  *
19  */
20 public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
21     //Reusable Text object to avoid creating new objects in loop
22     private Text word = new Text();
23     //Regex Pattern
24     private Pattern pattern = Pattern.compile("\\b[A-Z][a-z]+\\b");
25 /**
26  * The map method is called once per line of input.
27  * It finds all words matching the defined pattern and emits them with count 1.
28  */
29 @Override
30 public void map(LongWritable key, Text value, Context context)
31     throws IOException, InterruptedException {
32     // Apply regex matcher to the line
33     ...
28 */
29 @Override
30 public void map(LongWritable key, Text value, Context context)
31     throws IOException, InterruptedException {
32     // Apply regex matcher to the line
33     Matcher matcher = pattern.matcher(value.toString());
34     // For each match, emit the word with count 1
35     while (matcher.find()) {
36         // Extract matched word
37         word.set(matcher.group());
38         // Emit (word, 1)
39         context.write(word, new IntWritable(1));
40     }
41 }
42 }
43

```

Figure 7. The mapper class for emitting every word in the document with a frequency 1.



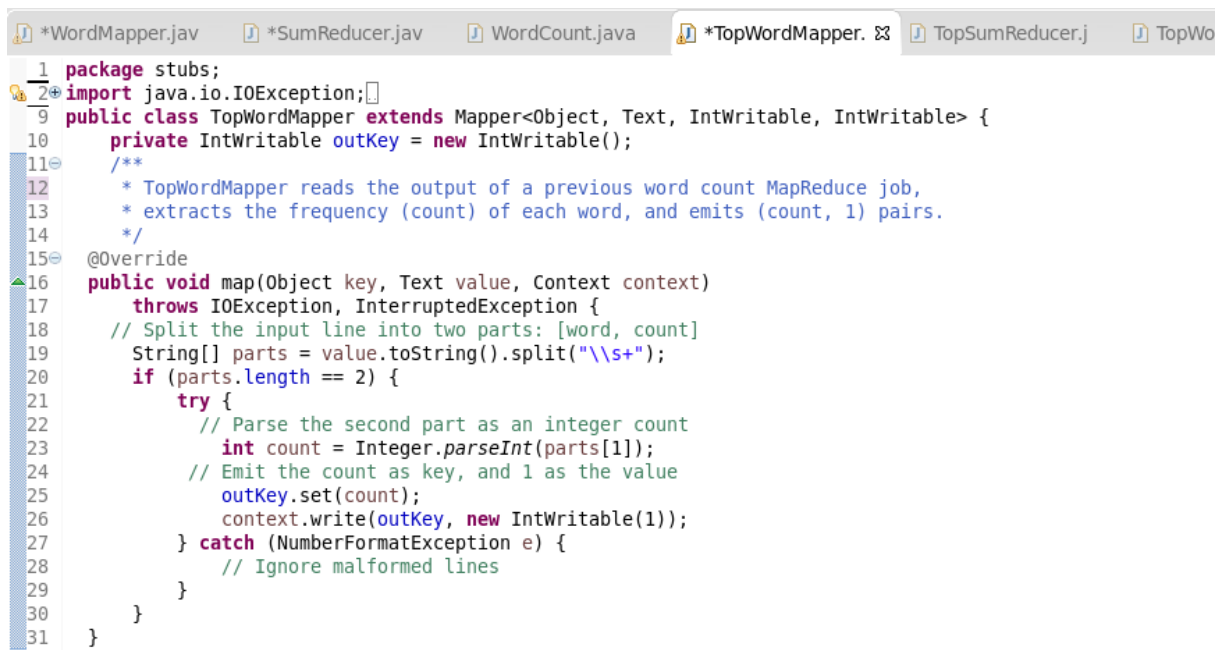
```

1 package stubs;
2
3 import java.io.IOException;
4
5 /**
6  * SumReducer receives (word, list of counts) and sums them.
7  * This is used in the first job to count total occurrences of each word.
8  */
9
10 public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
11     /**
12      * The reduce method is called once for each unique key (word).
13      * It sums up all the integer values (which are typically 1s from the mapper).
14      */
15     @Override
16     public void reduce(Text key, Iterable<IntWritable> values, Context context)
17         throws IOException, InterruptedException {
18         int wordCount = 0;
19         // Sum all the values associated with this word
20         for (IntWritable value : values) {
21             wordCount += value.get();
22         }
23         // Emit the word along with its total count
24         context.write(key, new IntWritable(wordCount));
25     }
26 }

```

Figure 8. The reducer class for aggregating word counts from the first job.

Second Job: Counting Single-Occurrence Terms: The second job reads the output of the first and emits (count, 1) pairs. The reducer filters for count == 1 and sums the total number of unique words that appeared exactly once.

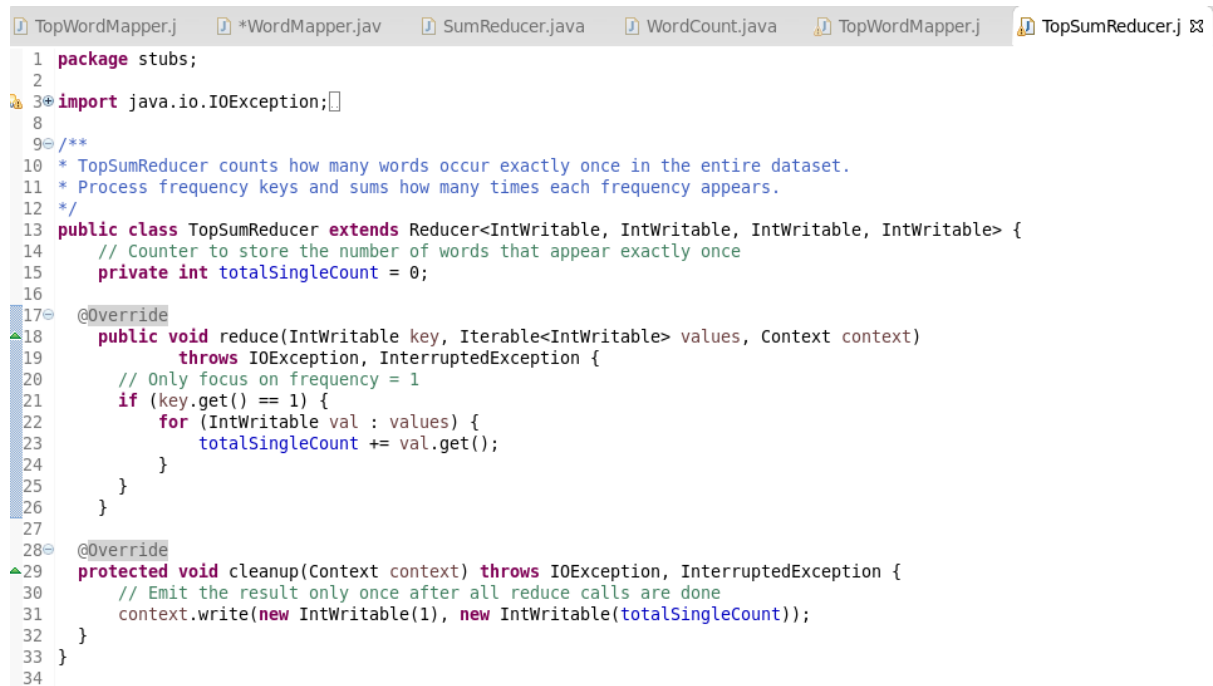


```

1 package stubs;
2 import java.io.IOException;
3
4 public class TopWordMapper extends Mapper<Object, Text, IntWritable, IntWritable> {
5     private IntWritable outKey = new IntWritable();
6
7     /**
8      * TopWordMapper reads the output of a previous word count MapReduce job,
9      * extracts the frequency (count) of each word, and emits (count, 1) pairs.
10     */
11
12     @Override
13     public void map(Object key, Text value, Context context)
14         throws IOException, InterruptedException {
15         // Split the input line into two parts: [word, count]
16         String[] parts = value.toString().split("\\s+");
17         if (parts.length == 2) {
18             try {
19                 // Parse the second part as an integer count
20                 int count = Integer.parseInt(parts[1]);
21                 // Emit the count as key, and 1 as the value
22                 outKey.set(count);
23                 context.write(outKey, new IntWritable(1));
24             } catch (NumberFormatException e) {
25                 // Ignore malformed lines
26             }
27         }
28     }
29 }

```

Figure 9. The mapper class for parsing (word, count) lines and emits (count, 1).



```

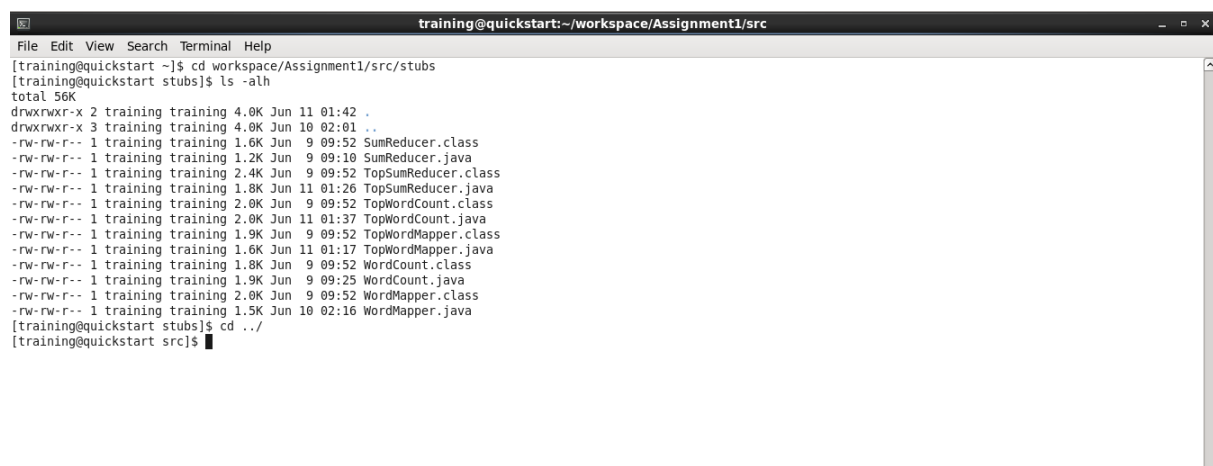
1 package stubs;
2
3 import java.io.IOException;
4
5 /**
6  * TopSumReducer counts how many words occur exactly once in the entire dataset.
7  * Process frequency keys and sums how many times each frequency appears.
8  */
9
10 public class TopSumReducer extends Reducer<IntWritable, IntWritable, IntWritable, IntWritable> {
11     // Counter to store the number of words that appear exactly once
12     private int totalSingleCount = 0;
13
14     @Override
15     public void reduce(IntWritable key, Iterable<IntWritable> values, Context context)
16         throws IOException, InterruptedException {
17         // Only focus on frequency = 1
18         if (key.get() == 1) {
19             for (IntWritable val : values) {
20                 totalSingleCount += val.get();
21             }
22         }
23     }
24
25     @Override
26     protected void cleanup(Context context) throws IOException, InterruptedException {
27         // Emit the result only once after all reduce calls are done
28         context.write(new IntWritable(1), new IntWritable(totalSingleCount));
29     }
30 }

```

Figure 10. The reducer class to sum up the number of words with count = 1.

4. List all commands used in this assignment in order, as well as readable execution screenshots with your name and student ID printed from the driver code. **[15 mark]**

To begin, the Java classes are compiled by first navigating to their directory using the command `cd workspace/Assignment1/src/stubs`. The command `ls -alh` is then used to verify that the required Java files are present in the directory. After confirming, the command `cd ../` is executed to return to the parent folder in preparation for the compilation step.



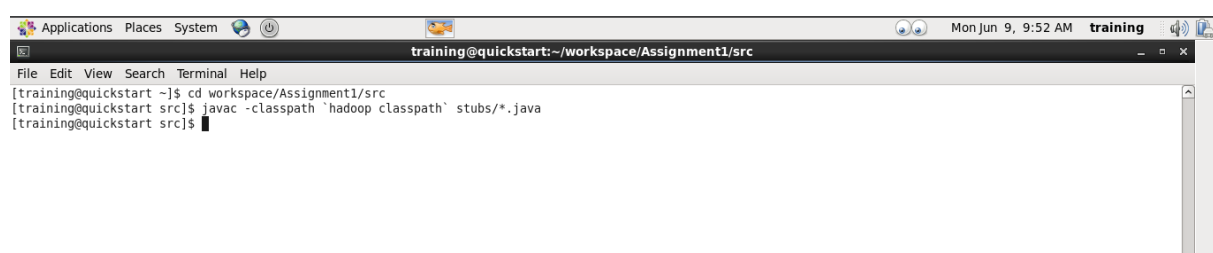
```

training@quickstart:~/workspace/Assignment1/src
File Edit View Search Terminal Help
[training@quickstart ~]$ cd workspace/Assignment1/src/stubs
[training@quickstart stubs]$ ls -alh
total 56K
drwxrwxr-x 2 training training 4.0K Jun 11 01:42 .
drwxrwxr-x 3 training training 4.0K Jun 10 02:01 ..
-rw-rw-r-- 1 training training 1.6K Jun 9 09:52 SumReducer.class
-rw-rw-r-- 1 training training 1.2K Jun 9 09:10 SumReducer.java
-rw-rw-r-- 1 training training 2.4K Jun 9 09:52 TopSumReducer.class
-rw-rw-r-- 1 training training 1.8K Jun 11 01:26 TopSumReducer.java
-rw-rw-r-- 1 training training 2.0K Jun 9 09:52 TopWordCount.class
-rw-rw-r-- 1 training training 2.0K Jun 11 01:37 TopWordCount.java
-rw-rw-r-- 1 training training 1.9K Jun 9 09:52 TopWordMapper.class
-rw-rw-r-- 1 training training 1.6K Jun 11 01:17 TopWordMapper.java
-rw-rw-r-- 1 training training 1.8K Jun 9 09:52 WordCount.class
-rw-rw-r-- 1 training training 1.9K Jun 9 09:25 WordCount.java
-rw-rw-r-- 1 training training 2.0K Jun 9 09:52 WordMapper.class
-rw-rw-r-- 1 training training 1.5K Jun 10 02:16 WordMapper.java
[training@quickstart stubs]$ cd ../
[training@quickstart src]$

```

Figure 11. The commands for navigating and checking the Java classes.

Once inside the correct directory, the Java classes located in the `stubs` folder are compiled using the command `javac -classpath `hadoop classpath` stubs/*.java`.



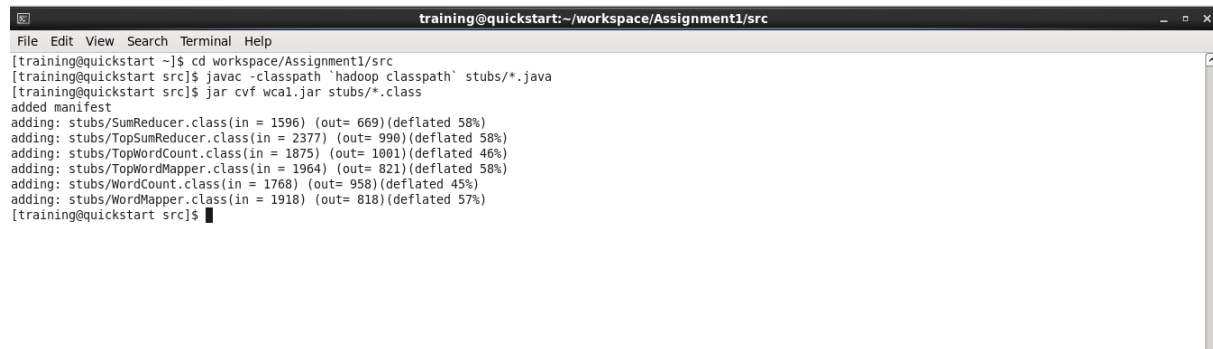
```

Applications Places System
training@quickstart:~/workspace/Assignment1/src
File Edit View Search Terminal Help
[training@quickstart ~]$ cd workspace/Assignment1/src
[training@quickstart src]$ javac -classpath `hadoop classpath` stubs/*.java
[training@quickstart src]$

```

Figure 12. The command for compiling the Java classes.

After successful compilation, the command `"jar cvf wca1.jar stubs/*.class"` is used to package all the compiled Java class files from the `"stubs"` folder into a JAR file named `"wca1.jar"`.



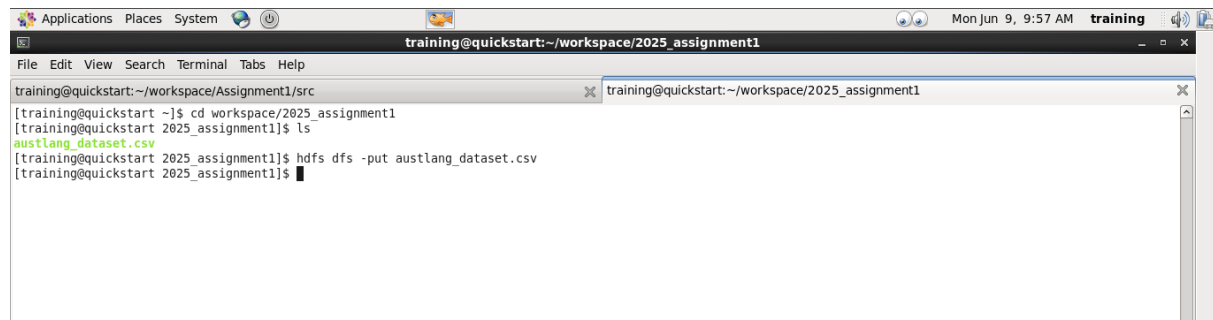
```

training@quickstart:~/workspace/Assignment1/src
[training@quickstart ~]$ cd workspace/Assignment1/src
[training@quickstart src]$ javac -classpath 'hadoop classpath' stubs/*.java
[training@quickstart src]$ jar cvf wca1.jar stubs/*.class
added manifest
adding: stubs/SumReducer.class(in = 1596) (out= 669)(deflated 58%)
adding: stubs/TopSumReducer.class(in = 2377) (out= 990)(deflated 58%)
adding: stubs/TopWordCount.class(in = 1875) (out= 1001)(deflated 46%)
adding: stubs/TopWordMapper.class(in = 1964) (out= 821)(deflated 58%)
adding: stubs/WordCount.class(in = 1768) (out= 958)(deflated 45%)
adding: stubs/WordMapper.class(in = 1918) (out= 818)(deflated 57%)
[training@quickstart src]$

```

Figure 13. The command for collecting compiled Java files into a JAR file.

Prior to running the MapReduce job, the dataset needs to be uploaded to HDFS. This is done by first navigating to the directory containing the dataset using the command `"cd workspace/2025_assignment1"`, and then executing `"hdfs dfs -put austlang_dataset.csv"` to transfer the dataset into the HDFS.



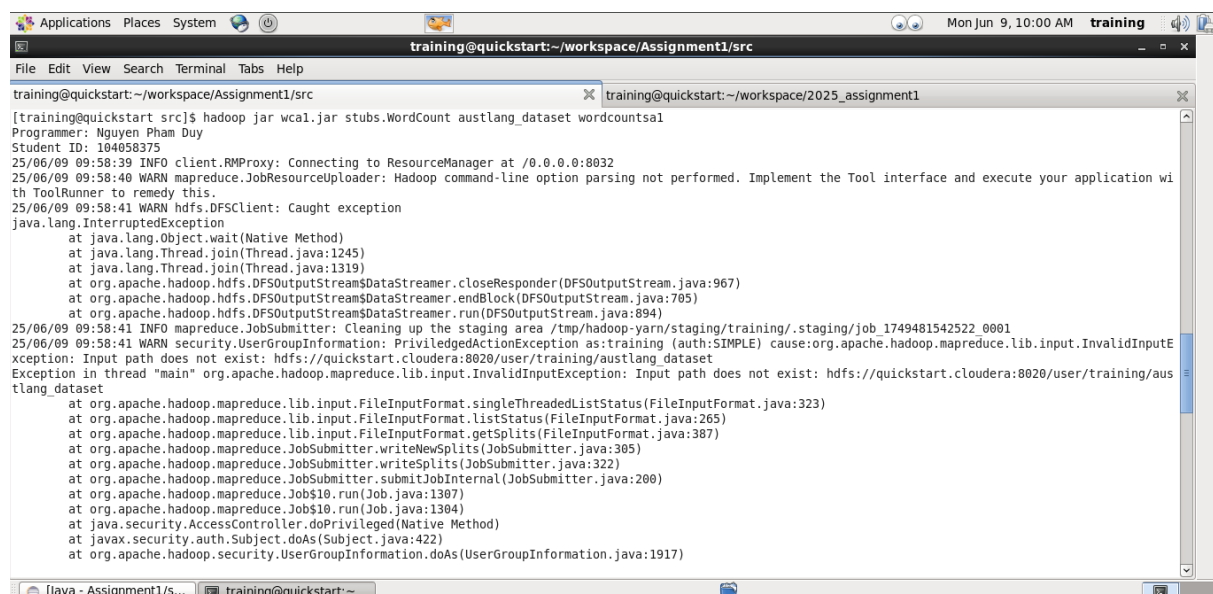
```

Applications Places System
training@quickstart:~/workspace/2025_assignment1
File Edit View Search Terminal Tabs Help
training@quickstart:~/workspace/Assignment1/src
[training@quickstart ~]$ cd workspace/2025_assignment1
[training@quickstart 2025_assignment1]$ ls
austlang_dataset.csv
[training@quickstart 2025_assignment1]$ hdfs dfs -put austlang_dataset.csv
[training@quickstart 2025_assignment1]$

```

Figure 14. The commands for putting the dataset into HDFS.

Once the dataset has been successfully uploaded to HDFS, navigate back to the directory containing the previously created JAR file. Then, initiate the first MapReduce job **"WordCount"** using the uploaded dataset as input and specify `"wordcounts1"` as the name of the output directory.



```

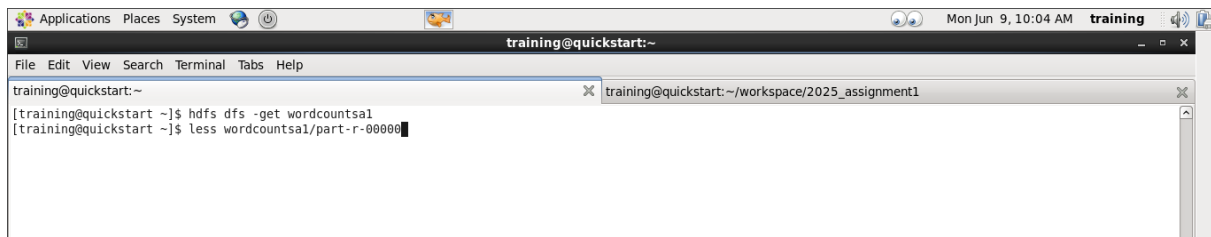
Applications Places System
training@quickstart:~/workspace/Assignment1/src
File Edit View Search Terminal Tabs Help
training@quickstart:~/workspace/Assignment1/src
[training@quickstart src]$ hadoop jar wca1.jar stubs.WordCount austlang_dataset wordcounts1
Programmer: Nguyen Pham Duy
Student ID: 104058375
25/06/09 09:58:39 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
25/06/09 09:58:40 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
25/06/09 09:58:41 WARN hdfs.DFSClient: Caught exception
java.lang.InterruptedException
    at java.lang.Object.wait(Native Method)
    at java.lang.Thread.join(Thread.java:1245)
    at java.lang.Thread.join(Thread.java:1319)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.closeResponder(DFSOutputStream.java:967)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.endBlock(DFSOutputStream.java:705)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.run(DFSOutputStream.java:894)
25/06/09 09:58:41 INFO mapreduce.JobSubmitter: Cleaning up the staging area /tmp/hadoop-yarn/staging/training/.staging/job_1749481542522_0001
25/06/09 09:58:41 WARN security.UserGroupInformation: PrivilegedActionException as:training (auth:SIMPLE) cause:org.apache.hadoop.mapreduce.lib.input.InvalidInputException: Input path does not exist: hdfs://quickstart.cloudera:8020/user/training/austlang_dataset
Exception in thread "main" org.apache.hadoop.mapreduce.lib.input.InvalidInputException: Input path does not exist: hdfs://quickstart.cloudera:8020/user/training/austlang_dataset
    at org.apache.hadoop.mapreduce.lib.input.FileInputFormat.singleThreadedListStatus(FileInputFormat.java:323)
    at org.apache.hadoop.mapreduce.lib.input.FileInputFormat.listStatus(FileInputFormat.java:265)
    at org.apache.hadoop.mapreduce.lib.input.FileInputFormat.get_splits(FileInputFormat.java:387)
    at org.apache.hadoop.mapreduce.JobSubmitter.writeNewSplits(JobSubmitter.java:305)
    at org.apache.hadoop.mapreduce.JobSubmitter.writeSplits(JobSubmitter.java:322)
    at org.apache.hadoop.mapreduce.JobSubmitter.submitJobInternal(JobSubmitter.java:200)
    at org.apache.hadoop.mapreduce.lib.input.FileInputFormat$1.run(Job.java:1307)
    at org.apache.hadoop.mapreduce.lib.input.FileInputFormat$1.run(Job.java:1304)
    at java.security.AccessController.doPrivileged(Native Method)
    at javax.security.auth.Subject.doAs(Subject.java:422)
    at org.apache.hadoop.security.UserGroupInformation.doAs(UserGroupInformation.java:1917)

```

Figure 15. The command for executing the first MapReduce job.

After the execution is complete, retrieve the output using the command `"hdfs dfs -get"`

wordcounts1", and then examine the results of the first MapReduce job by running "*less wordcounts1/part-r-00000*".

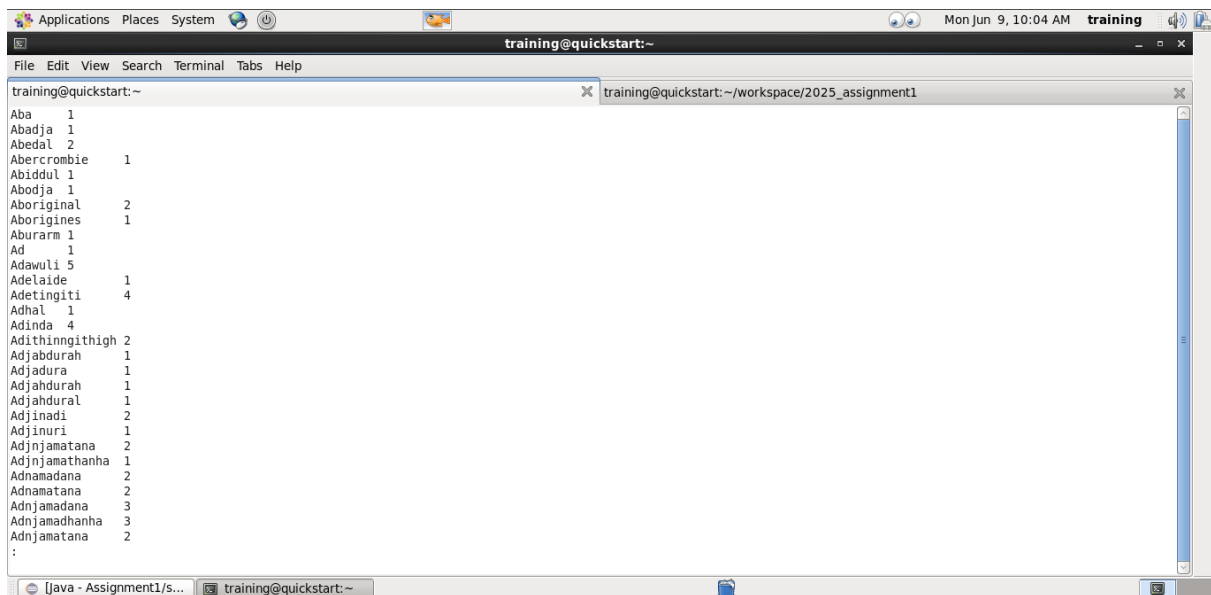


```

training@quickstart:~
[training@quickstart ~]$ hdfs dfs -get wordcounts1
[training@quickstart ~]$ less wordcounts1/part-r-00000

```

Figure 16. The commands for harvesting and checking the result of the first MapReduce job.



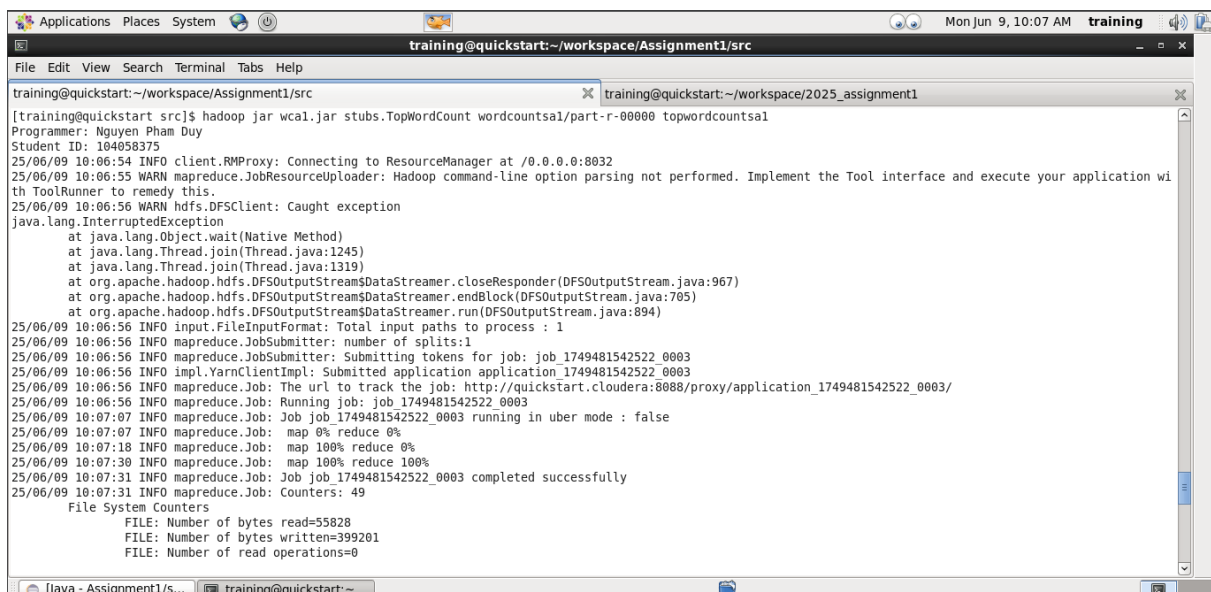
```

training@quickstart:~
Aba 1
Abadja 1
Abadal 2
Abercrombie 1
Abiddul 1
Abodja 1
Aboriginal 2
Aborigines 1
Aburarm 1
Ad 1
Adawuli 5
Adelaide 1
Adetintiti 4
Adhal 1
Adinda 4
Adithinngithigh 2
Adjabdurah 1
Adjadura 1
Adjahdurah 1
Adjahdural 1
Adjinadi 2
Adjinuri 1
Adjnjamatana 2
Adjnjamathanha 1
Adnamadana 2
Adnamatana 2
Adnjamadana 3
Adnjamadhanha 3
Adnjamatana 2
:

```

Figure 17. The result of the first MapReduce job.

Similar to the first MapReduce job, the second job is executed using the command "*hadoop jar wca1.jar stubs/TopWordCount wordcounts1/part-r-00000 topwordcounts1*". This runs the "*TopWordCount*" program, using the output from the first job (*wordcounts1/part-r-00000*) as input, and stores the results in a new output directory named "*topwordcounts1*".



```

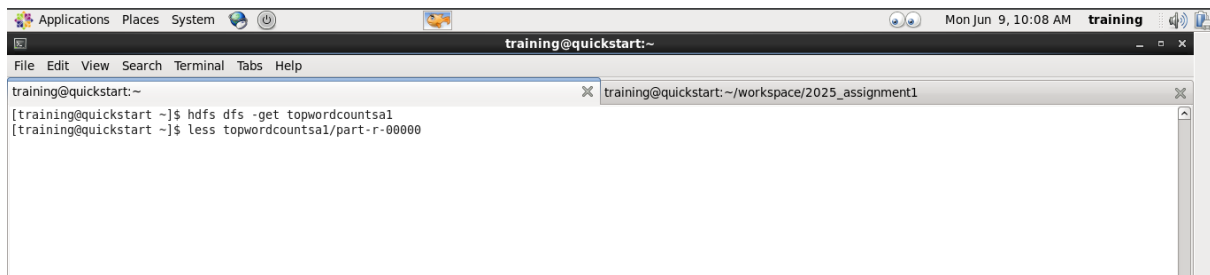
training@quickstart:~/workspace/Assignment1/src
[training@quickstart src]$ hadoop jar wca1.jar stubs.TopWordCount wordcounts1/part-r-00000 topwordcounts1
Programmer: Nguyen Pham Duy
Student ID: 104058375
25/06/09 10:06:54 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
25/06/09 10:06:55 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
25/06/09 10:06:56 WARN hdfs.DFSClient: Caught exception
java.lang.InterruptedException
    at java.lang.Object.wait(Native Method)
    at java.lang.Thread.join(Thread.java:1245)
    at java.lang.Thread.join(Thread.java:1319)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.closeResponder(DFSOutputStream.java:967)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.endBlock(DFSOutputStream.java:705)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.run(DFSOutputStream.java:894)
25/06/09 10:06:56 INFO input.FileInputFormat: Total input paths to process : 1
25/06/09 10:06:56 INFO mapreduce.JobSubmitter: number of splits:1
25/06/09 10:06:56 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1749481542522_0003
25/06/09 10:06:56 INFO impl.YarnClientImpl: Submitted application application_1749481542522_0003
25/06/09 10:06:56 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_1749481542522_0003/
25/06/09 10:06:56 INFO mapreduce.Job: Running job: job_1749481542522_0003
25/06/09 10:07:07 INFO mapreduce.Job: Job job_1749481542522_0003 running in uber mode : false
25/06/09 10:07:07 INFO mapreduce.Job: map 0% reduce 0%
25/06/09 10:07:18 INFO mapreduce.Job: map 100% reduce 0%
25/06/09 10:07:30 INFO mapreduce.Job: map 100% reduce 100%
25/06/09 10:07:31 INFO mapreduce.Job: Job job_1749481542522_0003 completed successfully
25/06/09 10:07:31 INFO mapreduce.Job: Counters: 49
File System Counters
  FILE: Number of bytes read=55828
  FILE: Number of bytes written=399201
  FILE: Number of read operations=0

```

Figure 18. The command for executing the second MapReduce job.

Once the second MapReduce job has completed successfully, retrieve the output using the

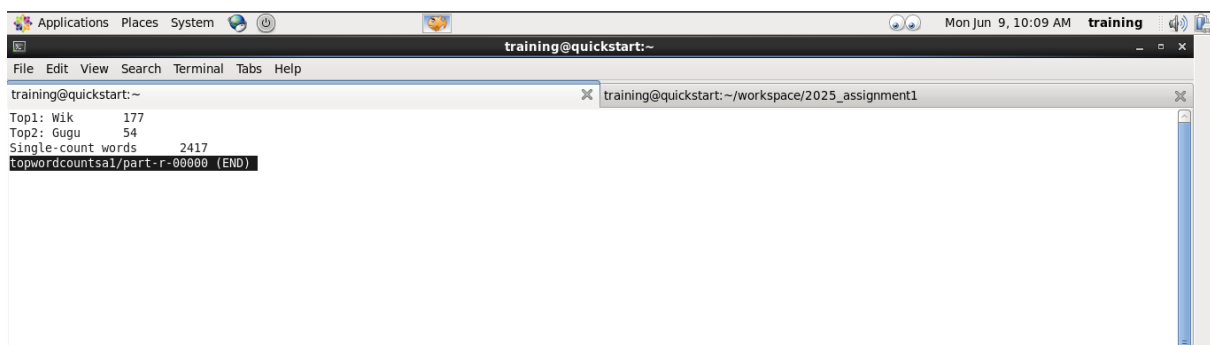
command `"hdfs dfs -get topwordcounts1"`, and view the results by running `"less topwordcounts1/part-r-00000"`.



```

training@quickstart:~
[training@quickstart ~]$ hdfs dfs -get topwordcounts1
[training@quickstart ~]$ less topwordcounts1/part-r-00000
  
```

Figure 19. The commands for harvesting and collecting the result of the second MapReduce job.

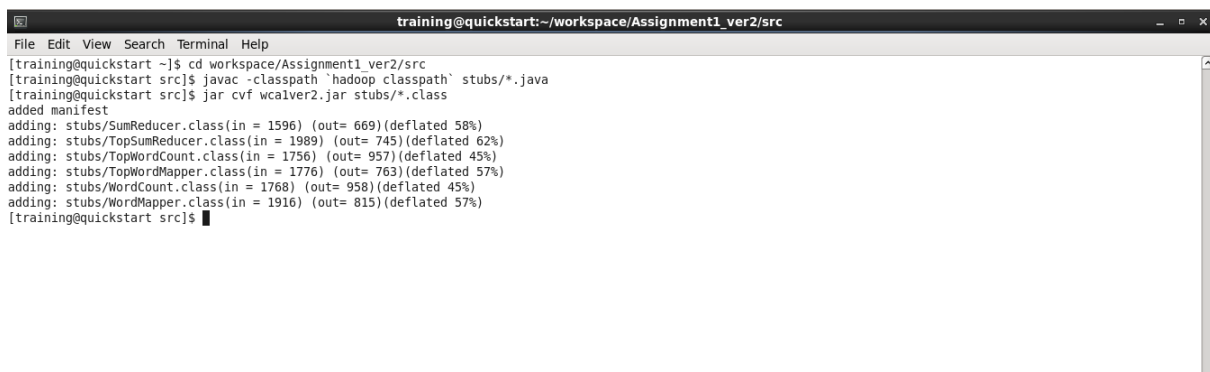


```

training@quickstart:~
Top1: Wik      177
Top2: Gugu     54
Single-count words      2417
topwordcounts1/part-r-00000 (END)
  
```

Figure 20. The result of the second MapReduce job.

(*)Additional MapReduce Implementation: Counting Terms that start with all upper-case letters and follow by the lower-case letters Appearing Only Once in the Entire Document:



```

training@quickstart:~/workspace/Assignment1_ver2/src
[training@quickstart ~]$ cd workspace/Assignment1_ver2/src
[training@quickstart src]$ javac -classpath `hadoop classpath` stubs/*.java
[training@quickstart src]$ jar cvf wcalver2.jar stubs/*.class
added manifest
adding: stubs/SumReducer.class(in = 1596) (out= 669)(deflated 58%)
adding: stubs/TopSumReducer.class(in = 1989) (out= 745)(deflated 62%)
adding: stubs/TopWordCount.class(in = 1756) (out= 957)(deflated 45%)
adding: stubs/TopWordMapper.class(in = 1776) (out= 763)(deflated 57%)
adding: stubs/WordCount.class(in = 1768) (out= 958)(deflated 45%)
adding: stubs/WordMapper.class(in = 1916) (out= 815)(deflated 57%)
[training@quickstart src]$
  
```

Figure 21. The commands to compile and collect the compiled Java files into a JAR file.

```

training@quickstart:~/workspace/Assignment1_ver2/src
File Edit View Search Terminal Help
[training@quickstart src]$ hadoop jar wcaver2.jar stubs.WordCount austlang_dataset.csv wordcountsalone
Programmer: Nguyen Pham Duy
Student ID: 104058375
25/06/10 10:17:48 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
25/06/10 10:17:49 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
25/06/10 10:17:50 INFO input.FileInputFormat: Total input paths to process : 1
25/06/10 10:17:50 WARN hdfs.DFSClient: Caught exception
java.lang.InterruptedException
    at java.lang.Object.wait(Native Method)
    at java.lang.Thread.join(Thread.java:1245)
    at java.lang.Thread.join(Thread.java:1319)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.closeResponder(DFSOutputStream.java:967)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.endBlock(DFSOutputStream.java:705)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.run(DFSOutputStream.java:894)
25/06/10 10:17:50 WARN hdfs.DFSClient: Caught exception
java.lang.InterruptedException
    at java.lang.Object.wait(Native Method)
    at java.lang.Thread.join(Thread.java:1245)
    at java.lang.Thread.join(Thread.java:1319)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.closeResponder(DFSOutputStream.java:967)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.endBlock(DFSOutputStream.java:705)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.run(DFSOutputStream.java:894)
25/06/10 10:17:50 INFO mapreduce.JobSubmitter: number of splits:1
25/06/10 10:17:51 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1749571477496_0001
25/06/10 10:17:52 INFO impl.YarnClientImpl: Submitted application application_1749571477496_0001
25/06/10 10:17:52 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8080/proxy/application_1749571477496_0001/
25/06/10 10:17:52 INFO mapreduce.Job: Running job: job_1749571477496_0001
25/06/10 10:18:12 INFO mapreduce.Job: Job job_1749571477496_0001 running in uber mode : false
25/06/10 10:18:12 INFO mapreduce.Job: map 0% reduce 0%
25/06/10 10:18:45 INFO mapreduce.Job: map 100% reduce 0%
25/06/10 10:18:58 INFO mapreduce.Job: map 100% reduce 100%
25/06/10 10:18:59 INFO mapreduce.Job: Job job_1749571477496_0001 completed successfully

```

Figure 22. The command for executing the first MapReduce on Hadoop.

```

training@quickstart:~
File Edit View Search Terminal Help
[training@quickstart ~]$ hdfs dfs -get wordcountsalone
[training@quickstart ~]$ less wordcountsalone/part-r-000000

```

Figure 23. The commands for harvesting and checking the result of the first MapReduce.

```

training@quickstart:~
File Edit View Search Terminal Help
Aba 1
Abadja 1
Abedal 2
Abercrombie 1
Abiddul 1
Abodja 1
Aboriginal 2
Aborigines 1
Aburarm 1
Ad 1
Adawuli 5
Adelaide 1
Adetingiti 4
Adhal 1
Adinda 4
Adithinngithigh 2
Adjabburah 1
Adjadura 1
Adjahdurah 1
Adjahdural 1
Adjinadi 2
Adjinuri 1
Adjnjamatana 2
Adjnjamathanha 1
Adnamadana 2
Adnamatana 2
Adnjamadana 3
Adnjamadhanha 3
Adnjamatana 2
Adnjamathanha 3
Adnjamathera 2
Adnjmadhanha 2
wordcountsalone/part-r-000000

```

Figure 24. The result of the first MapReduce.


```

training@quickstart:~/workspace/Assignment1_ver2/src
File Edit View Search Terminal Help
[training@quickstart src]$ hadoop jar wcalver2.jar stubs.TopWordCount wordcountsalonce/part-r-00000 topwordcountsalonce
Programmer: Nguyen Pham Duy
Student ID: 104058375
25/06/10 10:25:16 INFO client.RMPProxy: Connecting to ResourceManager at /0.0.0.0:8032
25/06/10 10:25:16 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with
h ToolRunner to remedy this.
25/06/10 10:25:17 INFO input.FileInputFormat: Total input paths to process : 1
25/06/10 10:25:17 WARN hdfs.DFSClient: Caught exception
java.lang.InterruptedException
    at java.lang.Object.wait(Native Method)
    at java.lang.Thread.join(Thread.java:1245)
    at java.lang.Thread.join(Thread.java:1319)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.closeResponder(DFSOutputStream.java:967)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.endBlock(DFSOutputStream.java:705)
    at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.run(DFSOutputStream.java:894)
25/06/10 10:25:17 INFO mapreduce.JobSubmitter: number of splits:1
25/06/10 10:25:17 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1749571477496_0002
25/06/10 10:25:18 INFO impl.YarnClientImpl: Submitted application application_1749571477496_0002
25/06/10 10:25:18 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_1749571477496_0002/
25/06/10 10:25:18 INFO mapreduce.Job: Running job: job_1749571477496_0002
25/06/10 10:25:27 INFO mapreduce.Job: Job job_1749571477496_0002 running in uber mode : false
25/06/10 10:25:27 INFO mapreduce.Job: map 0% reduce 0%
25/06/10 10:25:37 INFO mapreduce.Job: map 100% reduce 0%
25/06/10 10:25:51 INFO mapreduce.Job: map 100% reduce 100%
25/06/10 10:25:51 INFO mapreduce.Job: Job job_1749571477496_0002 completed successfully
25/06/10 10:25:51 INFO mapreduce.Job: Counters: 49
    File System Counters
        FILE: Number of bytes read=160716
        FILE: Number of bytes written=607871
        FILE: Number of read operations=0
        FILE: Number of large read operations=0
        FILE: Number of write operations=0
        HDFS: Number of bytes read=167196

```

Figure 25. The command for executing the second MapReduce on Hadoop.

```

training@quickstart:~/workspace/Assignment1_ver2/src
File Edit View Search Terminal Help
[training@quickstart ~]$ hdfs dfs -get topwordcountsalonce
[training@quickstart ~]$ less topwordcountsalonce/part-r-00000

```

Figure 26. The commands for harvesting and checking the result of the second MapReduce.

```

training@quickstart:~/workspace/Assignment1_ver2/src
File Edit View Search Terminal Help
1 6961
topwordcountsalonce/part-r-00000 (END)

```

Figure 27. The result of the second MapReduce.

5. You are assigned to find the word counts of any terms in the given dataset starting with the upper-case letter "A," "C," "G," "W," or "Y," followed by lowercase letters regardless of the length. Answering the questions:

- a. What is the term under the given criteria with the highest appearance count, and what is its count? **[10 marks]**

The term is "Wik", and its appearance count is 177.

- b. What is the term under the given criteria for the second high appearance count, and what is its count? **[10 marks]**

The term is "Gugu", and its appearance count is 54.

- c. How many terms only have a single count in the document? **[5 marks]**

There are 2417 terms that have a single count in the document under the given criteria, and there are 6961 terms only have a single count for all words start with a upper-case letter and follow by the lower-case letters in the whole document.

6. Video presentation with the required content. **[Pass/Fail]**

The video needs to be submitted independently on Canvas.